

# Problem Set #2: Build Your Personal Book Manager

---

## Goal

---

Build a small, personal command-line application for managing books.

The system should help you:

- keep track of books you own, want to read, are reading, or have finished
- search and browse your library
- enrich books with useful metadata
- generate personalized recommendations
- run several recommendation strategies in parallel
- combine and refine results through pipes
- show progress while work is happening
- provide a rich terminal interface using Gum

The purpose of this problem set is not to build a large application. The purpose is to build a **small system whose architecture you understand end to end**.

You may use Codex. However, you should be able to open any file in your project and explain what it does, what goes into it, what comes out of it, and how it connects to the rest of the application.

---

## Architectural Principle

---

Your application should be built from **small, understandable Bash programs**.

The architecture is:

```
UI → Workflows → Book / Recommendation Components → Data Layer → Storage
```

The application should not place everything in one Bash file.

Instead, each architectural responsibility should live in a separate file. The filesystem itself should make the architecture visible.

---

## Required Project Structure

---

```
book-manager/  
|  
├─ app.sh  
|  
├─ ui/  
|   ├─ main_menu.sh  
|   ├─ library_screen.sh  
|   └─ recommendations_screen.sh  
|  
├─ workflows/  
|   └─ manage_library.sh
```

```
|   └─ get_recommendations.sh
|
├─ books/
|   └─ fetch_book_metadata.sh
|       └─ search_books.sh
|
├─ recommendations/
|   └─ recommend_from_history.sh
|       └─ recommend_from_interests.sh
|           └─ recommend_for_discovery.sh
|               └─ refine_recommendations.sh
|
└─ data/
    └─ book_database.sh
        └─ books.csv
```

You may add files if you have a clear reason, but do not remove or collapse the required architectural layers.

---

## What Each File Must Do

---

### **app.sh**

The entry point for the application.

Responsibilities:

- start the application
- call the main UI
- connect the top-level pieces together
- remain small

**app.sh** should not contain database logic, recommendation logic, or large UI sections.

---

## UI Layer

---

### **ui/main\_menu.sh**

The application's main menu.

Use **Gum** to let the user choose actions such as:

- Browse Library
- Add Book
- Search Library
- Get Recommendations
- Quit

This file should focus on interaction, not application logic.

### **ui/library\_screen.sh**

Displays library-related information.

Possible responsibilities:

- show saved books
- show reading status
- display search results
- present book details

### `ui/recommendations_screen.sh`

Displays recommendation-related information.

Possible responsibilities:

- show progress while recommendation agents are running
- show the final shortlist
- let the user select or save a recommended book

---

## Workflow Layer

### `workflows/manage_library.sh`

Coordinates library operations.

Examples:

```
User Input → Metadata → Database
```

or

```
Search Request → Search Component → Results → UI
```

This file should coordinate components rather than perform every task itself.

### `workflows/get_recommendations.sh`

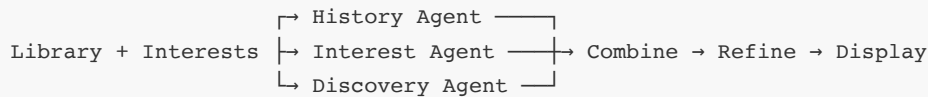
Coordinates the recommendation workflow.

This is where you should demonstrate **parallelization, synchronization, pipes, and streaming**.

At minimum:

1. Start the three recommendation programs in parallel.
2. Show that work is happening while they run.
3. Wait for them to finish.
4. Combine their outputs.
5. Pipe the combined recommendations into `refine_recommendations.sh`.
6. Send the final result to the UI.

Conceptually:



## Book Components

### **books/fetch\_book\_metadata.sh**

Takes basic book information and enriches it.

Possible input:

```
Dune | Frank Herbert
```

Possible output:

```
Dune | Frank Herbert | Science Fiction | 1965
```

Keep the interface simple and predictable.

You may use Codex as part of this step.

### **books/search\_books.sh**

Searches the user's library.

It should receive a search term and return matching books.

Examples:

```
./books/search_books.sh "history"
```

or through a pipe:

```
echo "history" | ./books/search_books.sh
```

## Recommendation Components

These three programs should represent **different ways of thinking about a recommendation**.

They should be independent so they can run in parallel.

### **recommendations/recommend\_from\_history.sh**

Recommend books based on what the user has already read, rated, or saved.

### **recommendations/recommend\_from\_interests.sh**

Recommend books based on the user's stated interests, topics, fields, hobbies, or goals.

### **recommendations/recommend\_for\_discovery.sh**

Recommend something intentionally outside the user's normal patterns.

The purpose is exploration rather than similarity.

### **recommendations/refine\_recommendations.sh**

Receives recommendation candidates through `stdin`.

Its job is to:

- remove obvious duplicates
- remove books already in the library
- reduce the candidate list
- polish or rank the final shortlist

It should produce a clean final result on `stdout`.

This makes it usable in a pipeline:

```
cat recommendations.txt | ./recommendations/refine_recommendations.sh
```

---

## Data Layer

### **data/book\_database.sh**

This is the **only application component that should directly read from or write to** `books.csv`.

Other files should ask the data layer to perform operations such as:

- add a book
- list books
- search books
- update status
- update rating
- check whether a book already exists

This creates an abstraction boundary between your application and its storage.

The rest of the application should not care whether the data is stored in CSV, SQLite, or something else.

### **data/books.csv**

The persistent storage for your library.

Keep the structure simple. For example:

```
title,author,genre,status,rating,link
```

You may extend the schema if your application needs additional fields.

---

## Required Technical Concepts

Your application must demonstrate all of the following:

## Bash Programs

The application must be composed primarily of small Bash programs.

## Pipes

At least one meaningful workflow must pass output from one program directly into another using `|`.

Example pattern:

```
Generate → Filter → Refine
```

## Parallelization

The three recommendation programs must run concurrently using Bash background processes.

You should use the concepts introduced in class:

```
&  
$!  
wait
```

## Streaming / Progress

The user should be able to tell that work is happening while longer-running tasks execute.

Keep this simple. A changing status line, elapsed timer, or messages such as `running` / `done` are sufficient.

## Gum

Use Gum to create a richer command-line experience.

At minimum, use it for the main menu and user selection.

## Codex

You may use Codex inside your application and while developing it.

However, keep the code small and understandable.

A good test is:

```
Can you explain every file in your project without asking Codex what it does?
```

---

## Personalization

Your Book Manager should reflect **you**.

The architecture is prescribed. The experience is not.

You decide:

- what metadata matters to you
- what counts as a useful recommendation

- what your recommendation agents optimize for
- how you organize your library
- what your interface looks like
- what additional feature would make the system genuinely useful to you

Your system should reveal something about your interests, preferences, or way of thinking.

---

## Keep It Small

---

Do not optimize for the largest application.

Optimize for:

```
Small files
Clear responsibilities
Simple interfaces
Visible data flow
Understandable architecture
```

If one Bash file becomes difficult to understand, ask whether it is doing more than one job.

---

## Deliverables

---

Submit your work through **GitHub**.

1. Create a repository in **your own GitHub account**. You may choose the repository name.
2. Push your complete `book-manager/` project to the repository.
3. Include a short `README.md` containing:
  - how to run the application
  - one paragraph describing your architecture
  - one paragraph explaining what you personalized
4. Include a **short narrated demo video** in the repository, or provide a clearly visible link to it from the `README.md`. The video should:
  - show the application running in the terminal
  - make the interface clearly visible
  - demonstrate **two or three operations**
  - include your narration explaining what you are doing and what the application is doing
  - be short: the goal is simply to let someone open your project and quickly understand what you built
5. Be prepared to explain any file in your project and trace one complete workflow from user input to output.

## How to Submit

Enter the URL of your GitHub repository in the class sign-up sheet under the column **Assignment No 2**:

<https://docs.google.com/spreadsheets/d/1ZewlG5udWWpk3Kdrab5yDbSWDV4gnH3liKobIAp7HG4/edit?usp=sharing>

Your GitHub repository URL is your homework submission.

---

# What Success Looks Like

---

A successful submission does not need to be large or sophisticated.

It should feel like **one coherent application assembled from small components**.

When someone opens your project directory, they should be able to understand the architecture before reading much of the code.

When someone runs it, they should see a personal book-management application that demonstrates the core ideas from class:

```
Input → Workflow → Decision → Intelligence → Output
```

combined with:

```
Small Programs + Pipes + Parallelization + Streaming + Composition
```